

Locations in the Code that May Cause Hazards and Solutions

Location 1: Data Hazard Caused by the Division Instruction (div)

```
continueCheck:  
    div $t3, $t4  
    mfhi $t7
```

At the label continueCheck, there is the instruction div \$t3, \$t4. In a 5 - stage pipeline of MIPS (Fetch, Decode, Execute, Memory, Write - Back), the division operation usually takes multiple cycles to complete. The subsequent instruction mfhi \$t7 depends on the result of the div instruction (after the div instruction is executed, the quotient is stored in the lo register and the remainder is stored in the hi register, and mfhi is used to read the value of the hi register), which leads to a RAW (Read - After - Write) data hazard. Because when the mfhi instruction tries to read the value of the hi register, the div instruction may not have written the result to the register yet.

Solutions:

Forwarding logic can be added to the pipeline. When it is detected that the source operand of the mfhi instruction depends on the result of the div instruction, the intermediate result generated in the execution stage of the div instruction is directly forwarded to the execution stage of the mfhi instruction, avoiding pipeline stalls. For example, add multiplexers in front of the ALU. When this dependency is detected, select the result of the execution stage of the div instruction as the input of the mfhi instruction instead of reading the possibly unupdated value from the register file.

Location 2: Control Hazard Caused by the Branch Instruction (beq)

```
beq $t0, $t1, notPrime
```

In the main function, there are multiple beq instructions, such as beq \$t0, \$t1, notPrime. In a 5 - stage pipeline, the branch decision can only be determined in the execution stage (it is necessary to compare the values of two registers to judge whether to jump), and before the branch decision is made, the pipeline has already prefetched subsequent instructions. If the branch condition is met, then the prefetched subsequent instructions need to be flushed, which leads to a control hazard. Because before the branch decision is made, we are not sure whether to continue executing the sequential instructions or the instructions after the jump, and the prefetched instructions may be on the wrong path.

Solutions:

Adopt branch prediction techniques. For example, use static branch prediction and assume that the branch does not occur (that is, predict that the condition of the beq instruction is not met), and continue to prefetch instructions in sequence. If the branch condition is ultimately met, then flush the instructions on the wrong path that have been prefetched and refetch from the correct address. More sophisticated dynamic branch prediction techniques can also be used, which predict whether a branch will occur based on the historical information of

program execution, improving the accuracy of prediction and reducing the performance loss caused by flushing instructions due to incorrect predictions.